

Single-Step SDE PPO for Flow-Matching Models

Bandit Reduction, Exact Credit Assignment, and a Per-Sample Design (Flow-Factory)

Flow-Factory

June 19, 2026

Abstract

Flow-Factory’s critic-based PPO currently rolls out a *full* SDE trajectory: every denoising step injects noise, and Generalized Advantage Estimation (GAE) distributes the terminal reward over all steps. This note develops an alternative regime in which *exactly one* denoising step k is stochastic (SDE) and all other steps are deterministic (ODE). We show this reduces the denoising MDP to a *contextual bandit* at step k : the state is the deterministic ODE-mapped latent $\mathbf{z}_k$, the action is the single SDE transition $\mathbf{z}_{k+1} \sim \pi_\theta(\cdot \mid \mathbf{z}_k)$, and the reward is a *deterministic* function of that action. The deterministic tail makes the action-value exact, $Q(\mathbf{z}_k, \mathbf{z}_{k+1}) = R$, so the advantage $A = R - V_\phi(\mathbf{z}_k)$ is *exact* (no bootstrap/GAE bias) and the policy gradient is a single clean term $\nabla \log \pi_\theta(\mathbf{z}_{k+1} \mid \mathbf{z}_k) (R - V_\phi)$. A variance decomposition shows that, conditioned on $\mathbf{z}_k$, all of R ’s randomness is due to the step- k noise, so the credit is unconfounded—the precise sense in which this scheme “assigns credit correctly.” We give the corrected advantage / value / critic-fitting logic, show it is the $S=1$ special case of the existing `compute_gae`, and present a concrete *per-sample* implementation design (scheduler per-sample SDE mask, single-transition storage, collapsed optimize loop) so that one rollout batch covers all timesteps. Implementation is deferred; this document is the theory + design reference. Companion: `docs/ppp/ppp_analysis` (the current full-SDE trainer) and `docs/ppp/ppp_papers_comparison`.

Contents

1	Motivation	2
2	Setup: the Denoising Chain with SDE and ODE Steps	2
3	Single-Step SDE Reduces Denoising to a Contextual Bandit	3
4	Exact Value and Advantage (Deterministic-Tail Identity)	4
5	Single-Step Policy Gradient and PPO Objective	4
6	Why the Credit Assignment Is Correct (Variance Decomposition)	5
7	Corrected Advantage / Value / Critic-Fitting Logic	5
8	Coverage: Per-Sample k as Randomized-Coordinate Policy Gradient	5
9	Train/Deploy Sampler Mismatch	6
10	Relation to Existing Methods	6

11 Implementation Design (Per-Sample SDE Step)	6
11.1 Scheduler: per-sample SDE mask + safe log-prob	6
11.2 Rollout: per-sample gate	7
11.3 Sample storage: a single transition per sample	7
11.4 Feedback and optimize: collapse to the bandit update	7
11.5 Hparams and config	7
11.6 Pseudocode	8
12 Summary	8

1 Motivation

In the current trainer (`docs/ppo/ppo_analysis`) the denoising rollout is a finite-horizon MDP over the SDE steps; *every* step is stochastic, and GAE spreads the single terminal reward R across all S steps. This is flexible but has two costs for *credit assignment*:

1. **Confounded per-step credit.** The per-step advantage $A_s = R - V_\phi(\mathbf{z}_s)$ (the $\gamma=\lambda=1$ case) treats one Monte-Carlo R as the return from $\mathbf{z}_s$, but R 's value depends on the noise of *all* downstream steps $s, \dots, S-1$. The signal attributable to the specific action at step s is buried in the variance contributed by the other steps. With $\lambda < 1$ the bootstrap $V_\phi(\mathbf{z}_{s+1})$ removes some variance at the price of bias.
2. **Every step trained every rollout.** The optimize loop iterates all S SDE steps (`trainers/ppo/trainer.py`), so compute scales with S .

Idea. Make exactly one step k stochastic and run the rest as ODE. Then the final reward's randomness comes *only* from step k , so the reward is causally and statistically attributable to that one action—exact credit assignment—and we train only that step (Flow-GRPO-style step reduction). Rotating k across samples/iterations trains the whole velocity field.

This is realizable today at the sampler level. Flow-Factory's scheduler already turns a step with `noise_level=0` into an exact ODE step: for Flow-SDE the transition mean collapses to $\mathbf{z}_{i+1} = \mathbf{z}_i + \mathbf{v}_\theta dt$ and no noise is injected (`scheduler/flow_match_euler_discrete.py`). Selecting a single SDE step per rollout (`num_sde_steps=1`) therefore already produces the single-SDE trajectory, and—as §7 shows—`compute_gae` already collapses to the exact single-step advantage. What is missing is (i) the theory that justifies and characterizes this regime, and (ii) a *per-sample* k so a single batch covers all timesteps (today the SDE step is one global choice per epoch).

2 Setup: the Denoising Chain with SDE and ODE Steps

A rollout produces latents $\mathbf{z}_0 \rightarrow \mathbf{z}_1 \rightarrow \dots \rightarrow \mathbf{z}_T$, with $\mathbf{z}_0 \sim \mathcal{N}(0, \mathbf{I})$ the initial noise and $\mathbf{z}_T$ decoded to the sample $\mathbf{x} = \text{dec}(\mathbf{z}_T)$ scored by the (terminal) reward $R = \sum_m w_m r_m(\mathbf{x})$. Step i uses the network's velocity prediction $\mathbf{v}_\theta(\mathbf{z}_i, t_i)$. Two transition types:

$$\text{ODE (deterministic): } \mathbf{z}_{i+1} = \mathbf{z}_i + \mathbf{v}_\theta(\mathbf{z}_i, t_i) dt_i, \quad (1)$$

$$\text{SDE (stochastic): } \mathbf{z}_{i+1} = \boldsymbol{\mu}_\theta(\mathbf{z}_i, t_i) + \sigma_i \boldsymbol{\epsilon}_i, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(0, \mathbf{I}), \quad (2)$$

where $\boldsymbol{\mu}_\theta$ is the Flow-SDE mean and $\sigma_i = \text{std_dev_t} \cdot \sqrt{-dt_i}$. Crucially, the scheduler's Flow-SDE mean satisfies $\boldsymbol{\mu}_\theta \rightarrow \mathbf{z}_i + \mathbf{v}_\theta dt_i$ and $\sigma_i \rightarrow 0$ as `noise_level` $\rightarrow 0$, so (2) *reduces exactly* to

(1) when noise is switched off. The per-step Gaussian density of (2),

$$\log \pi_\theta(\mathbf{z}_{i+1} \mid \mathbf{z}_i, t_i) = -\frac{\|\mathbf{z}_{i+1} - \boldsymbol{\mu}_\theta(\mathbf{z}_i, t_i)\|^2}{2\sigma_i^2} - \log \sigma_i - \frac{1}{2} \log 2\pi \quad (\text{per-dimension mean}), \quad (3)$$

is what the rollout stores and what the policy ratio recomputes; it is well-defined only when $\sigma_i > 0$ (an SDE step).

Single-step SDE. Fix one index $k \in \{0, \dots, T-1\}$. Use (2) at $i = k$ and (1) for all $i \neq k$. Write the deterministic ODE solution operators

$$\Phi_{0 \rightarrow k} : \mathbf{z}_0 \mapsto \mathbf{z}_k, \quad \Phi_{k+1 \rightarrow T} : \mathbf{z}_{k+1} \mapsto \mathbf{z}_T, \quad (4)$$

both deterministic functions of θ . Figure 1 contrasts the full-SDE and single-SDE trajectories.

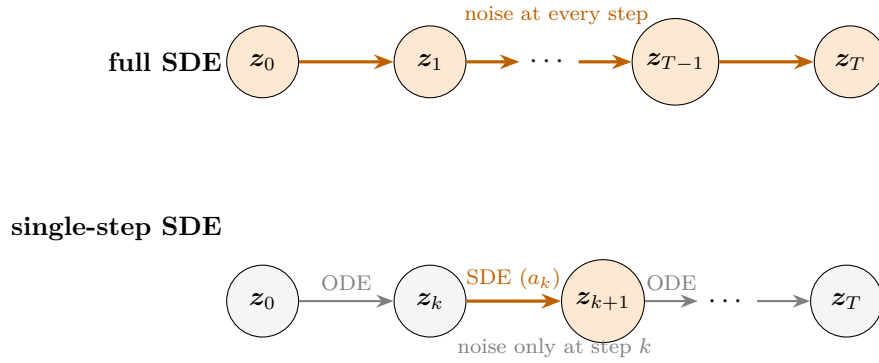


Figure 1: Top: the current full-SDE rollout (every transition stochastic). Bottom: the single-step SDE rollout—deterministic ODE except at step k , the sole source of randomness downstream of $\mathbf{z}_0$.

3 Single-Step SDE Reduces Denoising to a Contextual Bandit

With noise only at k , the trajectory has a single stochastic decision. Define the *contextual bandit*:

context (state)	$s = \mathbf{z}_k = \Phi_{0 \rightarrow k}(\mathbf{z}_0)$ (deterministic in $\mathbf{z}_0$)
action	$a = \mathbf{z}_{k+1} \sim \pi_\theta(\cdot \mid \mathbf{z}_k, t_k) = \mathcal{N}(\boldsymbol{\mu}_\theta, \sigma_k^2 \mathbf{I})$
reward	$R = \rho(\mathbf{z}_{k+1}) := r(\text{dec}(\Phi_{k+1 \rightarrow T}(\mathbf{z}_{k+1})))$ (deterministic in a)

The horizon- T MDP collapses to one decision: observe $\mathbf{z}_k$, draw the action $\mathbf{z}_{k+1}$, receive the deterministic reward $\rho(\mathbf{z}_{k+1})$. The initial noise $\mathbf{z}_0$ only enters through the context $\mathbf{z}_k$. Figure 2 summarizes.

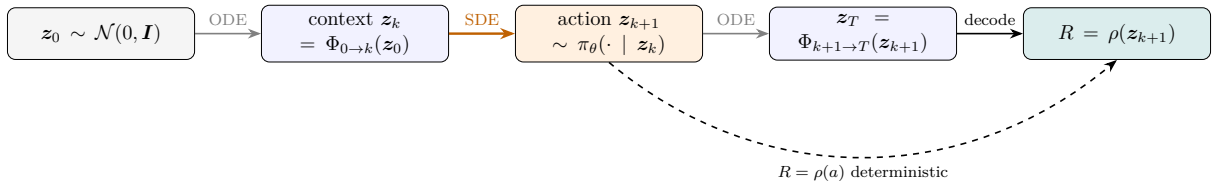


Figure 2: The single-step SDE process as a contextual bandit. Everything except the step- k transition is deterministic, so R is a deterministic function ρ of the action $\mathbf{z}_{k+1}$.

4 Exact Value and Advantage (Deterministic-Tail Identity)

Define the state and action values for this bandit:

$$Q^\pi(\mathbf{z}_k, a) = \mathbb{E}[R \mid \mathbf{z}_k, \mathbf{z}_{k+1} = a], \quad V_\phi(\mathbf{z}_k, t_k) \approx V^\pi(\mathbf{z}_k) = \mathbb{E}_{a \sim \pi_\theta(\cdot \mid \mathbf{z}_k)}[R]. \quad (5)$$

Proposition 1 (Deterministic tail $\Rightarrow$ exact advantage). *If steps $k+1, \dots, T-1$ are deterministic (ODE), then for the realized action $a = \mathbf{z}_{k+1}$,*

$$Q^\pi(\mathbf{z}_k, a) = R, \quad V^\pi(\mathbf{z}_{k+1}, t_{k+1}) = R,$$

and the action advantage is exact,

$$A(\mathbf{z}_k, a) = Q^\pi(\mathbf{z}_k, a) - V^\pi(\mathbf{z}_k) = R - V^\pi(\mathbf{z}_k, t_k),$$

with no bootstrap or GAE truncation error. Moreover $\mathbb{E}_{a \sim \pi_\theta(\cdot \mid \mathbf{z}_k)}[A(\mathbf{z}_k, a) \mid \mathbf{z}_k] = 0$.

Proof. Given $\mathbf{z}_{k+1}$, the tail $\mathbf{z}_T = \Phi_{k+1 \rightarrow T}(\mathbf{z}_{k+1})$ and $R = \rho(\mathbf{z}_{k+1})$ are deterministic, so $Q^\pi(\mathbf{z}_k, a) = \mathbb{E}[R \mid \mathbf{z}_{k+1} = a] = \rho(a) = R$. The post-action state $\mathbf{z}_{k+1}$ is terminal-equivalent: its value $V^\pi(\mathbf{z}_{k+1}) = \mathbb{E}[R \mid \mathbf{z}_{k+1}] = \rho(\mathbf{z}_{k+1}) = R$. The advantage is $Q - V = R - V^\pi(\mathbf{z}_k)$. Finally $\mathbb{E}_a[Q^\pi(\mathbf{z}_k, a)] = \mathbb{E}_a[R] = V^\pi(\mathbf{z}_k)$, so the advantage is mean-zero under the policy. $\square$

Contrast with full SDE, where $Q^\pi(\mathbf{z}_s, a_s) = \mathbb{E}[R \mid \mathbf{z}_s, a_s]$ still averages over the downstream noise $\epsilon_{s+1}, \dots, \epsilon_{S-1}$; using a single R as the return is a *high-variance* estimate of Q , and bootstrapping ($\lambda < 1$) trades that variance for bias. The deterministic tail removes both: R is Q .

5 Single-Step Policy Gradient and PPO Objective

Conditioned on $\mathbf{z}_0$, only the step- k transition carries a probability density; the ODE steps are deterministic (Dirac) and contribute no score term. Following the standard diffusion-policy-gradient construction (DDPO / Flow-GRPO), where the action is the next latent and the dynamics “ $s_{i+1} = a_i$ ” are parameter-free, the gradient is the single term

$$\nabla_\theta J_k(\theta) = \mathbb{E}_{\mathbf{z}_0, \epsilon_k} \left[\nabla_\theta \log \pi_\theta(\mathbf{z}_{k+1} \mid \mathbf{z}_k, t_k) (R - V_\phi(\mathbf{z}_k, t_k)) \right]. \quad (6)$$

The clipped PPO surrogate, with ratio $\rho_k = \exp(\log \pi_\theta(\mathbf{z}_{k+1} \mid \mathbf{z}_k) - \log \pi_{\text{old}}(\mathbf{z}_{k+1} \mid \mathbf{z}_k))$ and $\hat{A} = R - V_\phi$ (clamped / whitened), is

$$\mathcal{L}_k^{\text{pol}} = -\mathbb{E}[\min(\rho_k \hat{A}, \text{clip}(\rho_k, 1+c_-, 1+c_+) \hat{A})]. \quad (7)$$

Remark 1 (What this gradient does and does not capture). Like DDPO/Flow-GRPO, (6) treats the rollout latents as *given states* and differentiates only through the (single) stochastic transition; the cross-step pathwise dependence of the deterministic ODE steps on θ is intentionally not differentiated. Because $\mathbf{v}_\theta$ is shared across all noise levels, applying (6) at step k improves the policy slice $\mathbf{v}_\theta(\cdot, t_k)$; rotating k over training (§8) trains the whole field. The improvement over full-SDE is not the estimator family (still score-function with a baseline) but that the *within-step credit is exact* (Prop. 1).

6 Why the Credit Assignment Is Correct (Variance Decomposition)

Take the law of total variance for the single-SDE process, conditioning on the context $\mathbf{z}_k$:

$$\text{Var}(R) = \underbrace{\mathbb{E}_{\mathbf{z}_k}[\text{Var}(R \mid \mathbf{z}_k)]}_{\text{within-action (step } k)} + \underbrace{\text{Var}_{\mathbf{z}_k}(\mathbb{E}[R \mid \mathbf{z}_k])}_{\text{across contexts } (\mathbf{z}_0)}. \quad (8)$$

Conditioned on $\mathbf{z}_k$, $R = \rho(\boldsymbol{\mu}_\theta(\mathbf{z}_k) + \sigma_k \boldsymbol{\epsilon}_k)$ depends on a single random variable $\boldsymbol{\epsilon}_k$, so $\text{Var}(R \mid \mathbf{z}_k)$ is *entirely* the effect of the step- k action. The baseline $V_\phi(\mathbf{z}_k) = \mathbb{E}[R \mid \mathbf{z}_k]$ removes the second (across-context) term, leaving the advantage

$$\hat{A} = R - V_\phi(\mathbf{z}_k), \quad \mathbb{E}[\hat{A} \mid \mathbf{z}_k] = 0, \quad \text{Var}(\hat{A} \mid \mathbf{z}_k) = \text{Var}(R \mid \mathbf{z}_k), \quad (9)$$

which is a zero-mean, variance-minimal (baseline = conditional mean) and—critically— *unconfounded* estimate of how much this action beat the average at step k .

For full SDE, the analogous per-step quantity $R - V_\phi(\mathbf{z}_s)$ has $\text{Var}(R \mid \mathbf{z}_s) = \sum_{j \geq s} (\text{contribution of } \boldsymbol{\epsilon}_j)$: the credit for step s is mixed with the noise of all later steps. Single-step SDE collapses that sum to the one term that belongs to step k . This is the precise sense of “the reward’s return must come from this one SDE step.”

7 Corrected Advantage / Value / Critic-Fitting Logic

The single-SDE regime replaces the multi-step GAE machinery with a one-step bandit target. Concretely, per rollout sample i with SDE step k_i :

- **Advantage:** $\hat{A}_i = R_i - V_\phi(\mathbf{z}_{k_i}, t_{k_i})$, then (optionally) globally whitened across the batch, exactly as today (`normalize_advantage` $\rightarrow$ `whiten`). *No GAE recursion*; no γ, λ .
- **Value target (return):** $G_i = R_i$ (Monte-Carlo, and by Prop. 1 *exact*, not an approximation). Critic loss $\mathcal{L}_i^{\text{val}} = \frac{1}{2}(V_\phi(\mathbf{z}_{k_i}, t_{k_i}) - R_i)^2$, with the existing optional PPO value-clip around the old value (`value_clip_range`).
- **Critic semantics change.** The critic now learns the expected reward of a *single* exploratory step from $(\mathbf{z}, t)$ followed by deterministic integration (i.e. $V_\phi(\mathbf{z}, t) = \mathbb{E}[R \mid \mathbf{z}, t]$ under the single-SDE law). This differs from the full-SDE critic, which marginalizes over *all* downstream SDE noise; the single-SDE target has strictly lower variance.

It is the $S=1$ collapse of `compute_gae`. The existing `trainers/ppo/gae.py` already yields exactly this when there is one trained step. With values $V \in \mathbb{R}^{B \times 1}$ and terminal reward R , the backward recursion runs once at $s = 0 = S - 1$: $r = R$, $V_{\text{next}} = 0$, $\delta = R - V[:, 0]$, $A[:, 0] = \delta = R - V$, and $G = A + V = R$. So the corrected logic is not a new estimator but a *first-class* treatment of the single-step case (with per-sample k and a collapsed loop), which also removes the $\gamma=\lambda=1$ / contiguous-trajectory caveats that the full-SDE GAE documents.

8 Coverage: Per-Sample k as Randomized-Coordinate Policy Gradient

A fixed k trains only the slice $\mathbf{v}_\theta(\cdot, t_k)$. Sampling k (per sample, or per prompt-group) from a distribution q over steps and optimizing the mixture

$$\max_{\theta} \mathbb{E}_{k \sim q}[J_k(\theta)] \quad (10)$$

covers all timesteps. Because each rollout’s gradient (6) updates the step it sampled, this is a *randomized-coordinate* policy gradient over denoising steps: one batch with per-sample k_i touches every coordinate, keeping both the policy and the critic (§7) fresh across all noise levels. We recommend $q = \text{Unif}\{0, \dots, T - 2\}$ (exclude the final step, which the scheduler already treats as the deterministic readout) unless reward sensitivity is known to concentrate at particular noise levels, in which case q can be tilted.

Remark 2 (Why per-sample, not per-epoch). Today `current_sde_steps/train_timesteps` is one global choice per epoch (seeded by `epoch+seed`), so a single-SDE epoch would train one timestep and the critic would see one t per epoch. Per-sample k_i restores full timestep coverage each batch, which is both statistically (lower-variance field estimate) and practically (fresh critic baseline) preferable.

9 Train/Deploy Sampler Mismatch

Single-step SDE is an *exploration* distribution: it perturbs the ODE trajectory at one point to obtain a gradient, but deployment typically uses pure ODE (or a chosen SDE schedule). The marginal law of $\mathbf{z}_T$ under “SDE-at- k ” differs from full-ODE and full-SDE sampling. This is benign for fine-tuning the shared field $\mathbf{v}_\theta$ —the perturbation is local and, as the noise level $\rightarrow 0$, vanishes—but two points deserve care: (i) the exploration scale σ_k (`noise_level`) controls a bias–variance trade-off (too small $\Rightarrow$ weak signal; too large $\Rightarrow$ states far from the deployment manifold); (ii) as the policy drifts, the single-step perturbation should remain “consistent” with the ODE flow it perturbs—the same consistency concern formalized by DiffAC [5]. In practice, choosing k uniformly and a moderate `noise_level` keeps the exploration close to the deployment trajectory.

10 Relation to Existing Methods

- **Flow-GRPO** [1]: also reduces trained steps by subsampling SDE steps, but uses a *group-relative* (critic-free) advantage broadcast to those steps. Single-step SDE PPO uses a *learned per-state* baseline $V_\phi(\mathbf{z}_k)$ and an *exact single-action* advantage; with one SDE step the “which steps to train” question becomes “which one step,” answered per sample.
- **Current full-SDE PPO** (`docs/ppp/ppp_analysis`): GAE over all steps; value marginalizes all downstream noise; per-step credit via bootstrap. Single step makes $A = R - V_\phi$ exact and drops the multi-step loop.
- **Perturbed-state critic follow-up** (`docs/ppp/ppp_analysis`, §“DiffAC-style perturbed-state critic”): complementary—it widens *critic* coverage by re-noising clean samples, and can be combined here to keep V_ϕ accurate across all t even when the *policy* trains a single step.

11 Implementation Design (Per-Sample SDE Step)

This section specifies the code changes; **they are deferred** (to be implemented after this theory is reviewed). The design keeps the existing rollout/storage where possible and adds a per-sample SDE step plus a collapsed (bandit) optimize path.

11.1 Scheduler: per-sample SDE mask + safe log-prob

Files under `scheduler/`: `flow_match_euler_discrete.py`, the mirror `unipc_multistep.py`, and the contract `abc.py`.

- Hold a per-sample step assignment $\mathbf{k} \in \{0, \dots, T-1\}^B$ (a buffer set per rollout). Add a setter, e.g. `set_per_sample_sde_steps(k)`.
- Make `get_noise_level_for_timestep(t)` return a $(B,)$ *tensor*: `noise_level` where $t = t_{k_i}$ else 0. The Flow-SDE `step()` already broadcasts a per-sample `noise_level` via `to_broadcast_tensor`, so the mean/noise math in (2) is unchanged (samples with 0 collapse to ODE).
- **Safe log-prob.** (3) divides by σ_i ; for non-SDE samples $\sigma_i=0$. Guard the division (e.g. compute on a safe denominator and mask the non-SDE rows to 0/NaN-free) so a batched step with mixed SDE/ODE rows does not divide by zero.

11.2 Rollout: per-sample gate

Files: the shared inference path and per-adapter loops (e.g. `models/stable_diffusion/sd3_5.py`). Today the loop computes a scalar `current_noise_level = get_noise_level_for_timestep(t)` and gates `current_compute_log_prob = compute_log_prob` and `current_noise_level > 0`. With per-sample k , every step has *some* SDE rows, so:

- Replace the scalar gate by a per-sample-aware one (compute log-prob whenever *any* row is SDE at this step), ideally via a small scheduler helper (`should_compute_log_prob(t)`) to minimize per-adapter churn (constraint: changes to shared infra affect all adapters).
- Continue storing the full `all_latents`; store per-step `log_probs` with the non-SDE rows masked. Each sample contributes exactly one real log-prob (at its k_i).

11.3 Sample storage: a single transition per sample

Rather than rely on the shared `latent_index_map` (which assumes a common set of trajectory positions across the batch), gather *per sample* the one transition it trains on: $(\mathbf{z}_{k_i}, t_{k_i}, \mathbf{z}_{k_i+1}, \log \pi_{\text{old}}(\mathbf{z}_{k_i+1} | \mathbf{z}_{k_i}))$ plus the scalar R_i . This is a fixed-size per-sample record (independent of k_i), simplifying the feedback/optimize code.

11.4 Feedback and optimize: collapse to the bandit update

Files: `trainers/ppo/trainer.py`, `trainers/ppo/gae.py`.

- `prepare_feedback`: compute R_i (reuse `_compute_terminal_rewards`); $V_i^{\text{old}} = V_\phi(\mathbf{z}_{k_i}, t_{k_i})$ (one query per sample, replacing the per-step `_compute_old_values` loop); $\hat{A}_i = R_i - V_i^{\text{old}}$ then `whiten`; store $\hat{A}_i$, $G_i = R_i$, V_i^{old} as *per-sample scalars* (replacing the (B, S) tensors and the `compute_gae` call).
- `optimize`: drop the `for timestep_index in sorted_ts` loop. Per micro-batch, one PPO-clip update (7) at each sample's $(\mathbf{z}_{k_i}, t_{k_i})$ using its stored $\mathbf{z}_{k_i+1}$, and one critic step $V_\phi(\mathbf{z}_{k_i}, t_{k_i}) \rightarrow R_i$. The critic warmup / re-warmup paths (`_fit_critic_only`) collapse identically (one value query per sample).
- GAS bookkeeping: `get_num_train_timesteps` returns 1 trained step per sample (instead of `num_sde_steps`); update the auto-GAS multiplier accordingly.

11.5 Hparams and config

Files: `hparams/scheduler_args.py` and `hparams/training_args/ppo.py`; example under `examples/ppo/lora/sd3_5/`.

- Add `sde_step_selection`: `{global, per_sample}` (default `global` preserves current behavior) and the per-sample k distribution q (e.g. `uniform`); fail-fast validation: per-sample mode requires `dynamics_type=Flow-SDE` and a single SDE step per sample.
- Add an example `examples/ppo/lora/sd3_5/single_sde.yaml` (`num_inference_steps` unchanged; one SDE step per sample; advantage whitening on; value-clip optional).

11.6 Pseudocode

Algorithm 1 Per-sample single-step SDE PPO (one optimize round)

- 1: **Rollout**: sample $k_i \sim q$ per sample; run T -step generation with ODE everywhere except step k_i for sample i (per-sample `noise_level` mask).
 - 2: store per sample: $\mathbf{z}_{k_i}, t_{k_i}, \mathbf{z}_{k_i+1}, \log \pi_{\text{old}}(\mathbf{z}_{k_i+1} \mid \mathbf{z}_{k_i})$; decode $\mathbf{z}_T^{(i)}$.
 - 3: **Reward**: $R_i \leftarrow$ terminal weighted-sum score of $\text{dec}(\mathbf{z}_T^{(i)})$.
 - 4: **Feedback (no grad)**: $V_i^{\text{old}} \leftarrow V_\phi(\mathbf{z}_{k_i}, t_{k_i})$; $\hat{A}_i \leftarrow R_i - V_i^{\text{old}}$; $\hat{A} \leftarrow \text{WHITEN}(\hat{A})$; $G_i \leftarrow R_i$.
 - 5: **for** inner epoch, micro-batch **do**
 - 6: **Critic**: $V_i \leftarrow V_\phi(\mathbf{z}_{k_i}, t_{k_i})$; $\mathcal{L}^{\text{val}} = \frac{1}{2}(V_i - G_i)^2$ (optional value-clip); backward; step critic opt.
 - 7: **Policy**: recompute $\log \pi_\theta(\mathbf{z}_{k_i+1} \mid \mathbf{z}_{k_i})$; $\rho_i = \exp(\log \pi_\theta - \log \pi_{\text{old}})$;
 - 8: $\mathcal{L}^{\text{pol}} = -\min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1+c_-, 1+c_+) \hat{A}_i)$; backward; step policy opt.
 - 9: **end for**
-

Note the inner body has *no* timestep loop—one transition per sample—which is the compute reduction relative to the full-SDE loop over `sorted_ts`.

12 Summary

Injecting SDE noise at a single denoising step turns diffusion RL into a contextual bandit whose deterministic tail makes the advantage $A = R - V_\phi(\mathbf{z}_k)$ *exact* (Prop. 1) and whose reward variance, conditioned on the context, is attributable solely to that step (§6). The corrected logic is the $S=1$ collapse of our existing GAE, made first-class with per-sample k for full timestep coverage (§8). The implementation (§11) is a per-sample SDE mask in the scheduler plus a collapsed bandit optimize path, deferred to a follow-up after this theory is reviewed.

References

- [1] Jie Liu et al. Flow-GRPO: Training Flow-Matching Models via Online RL. arXiv:2505.05470, 2025. <https://arxiv.org/abs/2505.05470>
- [2] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, Sergey Levine. Training Diffusion Models with Reinforcement Learning (DDPO). arXiv:2305.13301, 2023. <https://arxiv.org/abs/2305.13301>
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>
- [4] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438, 2015. <https://arxiv.org/abs/1506.02438>

- [5] Xiangxin Zhou, Liang Wang, Yichi Zhou. Stabilizing Policy Gradients for SDEs via Consistency with Perturbation Process (DiffAC). ICML 2024; arXiv:2403.04154. <https://arxiv.org/abs/2403.04154>